

LAB #14

AGGREGATION IN MONGODB

Objectives:

- To understand the concept of Aggregation in MongoDB.
- To learn and apply various Aggregation Pipeline stages.

Theory:

What is Aggregation?

Aggregation in MongoDB is a powerful framework used to process data records and return computed results. It works similarly to SQL's GROUP BY clause, but is far more flexible. Aggregation operations group values from multiple documents together and can perform a variety of operations on the grouped data to return a single result.

MongoDB provides three ways to perform aggregation: the aggregation pipeline, the map-reduce function, and single purpose aggregation methods. The aggregation pipeline is the most commonly used approach.

Aggregation Pipeline

The aggregation pipeline is a framework for data aggregation modeled on the concept of data processing pipelines. Documents enter a multi-stage pipeline that transforms them into aggregated results. Each stage transforms the documents as they pass through.

Syntax:

```
db.collection.aggregate([  
  
  { stage1 },  
  
  { stage2 },  
  
  { stage3 },  
  
  ...  
  
])
```

Common Aggregation Pipeline Stages:

Stage	Description
\$match	Filters documents (like WHERE in SQL)
\$group	Groups documents by a specified key and applies accumulators
\$sort	Sorts documents in ascending or descending order
\$project	Reshapes documents (include/exclude fields)
\$limit	Limits the number of documents returned
\$skip	Skips a specified number of documents
\$unwind	Deconstructs an array field into multiple documents
\$lookup	Performs a left outer join with another collection
\$count	Counts the number of documents in the pipeline
\$avg / \$sum / \$min / \$max	Arithmetic accumulator operators used inside \$group

Aggregation Examples

1. \$match and \$group

Consider a collection named 'orders' with fields: item, quantity, price, and status. The following query groups orders by item and calculates the total quantity for each item.

```
> db.orders.aggregate([
  { $match: { status: "A" } }
])
< {
  _id: ObjectId('6a1d00b4f0b7482955417096'),
  item: 'almonds',
  quantity: 408,
  price: 120,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b7482955417097'),
  item: 'pecans',
  quantity: 162,
  price: 80,
  status: 'A'
}
{
```

```

> db.orders.aggregate([
  { $group: { _id: "$item", totalQty: { $sum: "$quantity" } } }
])
< {
  _id: 'almonds',
  totalQty: 458
}
{
  _id: 'pecans',
  totalQty: 262
}
{
  _id: 'bread',
  totalQty: 75
}

```

2. \$sort and \$limit

```

> db.orders.aggregate([
  { $sort: { price: -1 } }
])
< {
  _id: ObjectId('6a1d00b4f0b7482955417096'),
  item: 'almonds',
  quantity: 408,
  price: 120,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b7482955417099'),
  item: 'almonds',
  quantity: 50,
  price: 120,
  status: 'B'
}

```

```

{
  _id: ObjectId('6a1d00b4f0b7482955417097'),
  item: 'pecans',
  quantity: 162,
  price: 80,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b748295541709a'),
  item: 'pecans',
  quantity: 100,
  price: 80,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b7482955417098'),
  item: 'bread',
  quantity: 75,
  price: 30,
  status: 'B'
}

```

```
> db.orders.aggregate([
  { $limit: 3 }
])
```

```
< {
  _id: ObjectId('6a1d00b4f0b7482955417096'),
  item: 'almonds',
  quantity: 408,
  price: 120,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b7482955417097'),
  item: 'pecans',
  quantity: 162,
  price: 80,
  status: 'A'
}
{
  _id: ObjectId('6a1d00b4f0b7482955417098'),
  item: 'bread',
  quantity: 75,
  price: 30,
  status: 'B'
}
```

3. \$project

```
> db.orders.aggregate([
  { $project: { _id: 0, itemName: "$item", qty: "$quantity" } }
])
< {
  itemName: 'almonds',
  qty: 408
}
{
  itemName: 'pecans',
  qty: 162
}
{
  itemName: 'bread',
  qty: 75
}
{
  itemName: 'almonds',
  qty: 50
}
{
  itemName: 'pecans',
  qty: 100
}
```

4: \$unwind

Consider a students collection where each document has an array of scores. \$unwind deconstructs the array into separate documents:

```
> db.students.insertMany([
  { name: "Habiba", scores: [85, 90, 78] },
  { name: "Zain", scores: [70, 88, 92] }
])

db.students.aggregate([
  { $unwind: "$scores" }
])
```

```
{
  _id: ObjectId('6a1d0118f0b748295541709c'),
  name: 'Habiba',
  scores: 70
}
{
  _id: ObjectId('6a1d0118f0b748295541709c'),
  name: 'Habiba',
  scores: 88
}
{
  _id: ObjectId('6a1d0118f0b748295541709c'),
  name: 'Habiba',
  scores: 92
}
{
  _id: ObjectId('6a1d033df0b74829554170a0'),
  name: 'Habiba',
  scores: 85
}
```

```

{
  _id: ObjectId('6a1d033df0b74829554170a1'),
  name: 'Zain',
  scores: 70
}
{
  _id: ObjectId('6a1d033df0b74829554170a1'),
  name: 'Zain',
  scores: 88
}
{
  _id: ObjectId('6a1d033df0b74829554170a1'),
  name: 'Zain',
  scores: 92
}

```

5: \$lookup (Join)

\$lookup performs a left outer join to combine documents from two collections. Suppose we have a 'products' collection:

```

> db.orders.aggregate([
  { $lookup: {
    from: "products",
    localField: "item",
    foreignField: "name",
    as: "productDetails"
  }}
])
< {
  _id: ObjectId('6a1d00b4f0b7482955417096'),
  item: 'almonds',
  quantity: 408,
  price: 120,
  status: 'A',
  productDetails: [
    {
      _id: ObjectId('6a1d0183f0b748295541709d'),
      name: 'almonds',
      category: 'nuts'
    }
  ]
}

```

Lab Tasks:

1. Create a database named 'university' with a 'students' collection. Insert at least 8 student documents with fields: name, department, cgpa, semester, and courses (array).
2. Use the \$group stage to find the average CGPA per department.
3. Use \$match and \$group to count how many students in each department have a CGPA greater than 3.0.
4. Use \$unwind on the 'courses' array to list each course separately, then use \$group to count how many students are enrolled in each course.
5. Use \$sort and \$limit to find the top 3 students with the highest CGPA.
6. Use \$project to display only the name and cgpa fields, renaming cgpa to 'GradePoint'.
7. Create a second collection 'teachers' and use \$lookup to join it with 'students' on the department field.

Reflection Questions:

- What is the difference between aggregation and a simple find() query in MongoDB?
- What was the most challenging part of this lab, and how did you overcome it?